Vehicle Selector Pro

A Production-Grade Shopify App for Vehicle Fitment

Project Report · August 2026

Executive Summary

Vehicle Selector Pro is a complete Ruby on Rails application that lets Shopify merchants map vehicle attributes (Year, Make, Model, Trim, Engine) to their products and gives customers a dynamic, cascading fitment widget on the storefront. It ships as a production-grade Rails app with a normalized local data cache, Shopify Metafield sync, a Theme App Extension widget, App Proxy endpoints, GraphQL data integration, and asynchronous processing via ActiveJob and Sidekiq.

Live Demonstrations

Live demo (storefront): https://vehicle-selector-pro.fly.dev/demo

Live demo (merchant admin): https://vehicle-selector-pro.fly.dev/demo/admin

Source repository: https://github.com/ai-dev-2024/vehicle-selector-pro

GitHub repository homepage: https://github.com/ai-dev-2024/vehicle-selector-pro

Feature Highlights

1. Cascading storefront widget

Year, Make, Model, Trim, Engine dropdowns driven by HMAC-SHA256-signed App Proxy queries, serving only the fitment data relevant to each selection.

2. Product fitment badges

Guaranteed Exact Fit / Does NOT Fit / Universal Fit badges rendered on product pages so shoppers instantly know whether a part fits their vehicle.

3. My Garage

Shoppers save multiple vehicles and switch between them across pages — a real retention feature, not just a filter.

4. Merchant admin dashboard

Fitment matrix, vehicle library, bulk CSV import, sync monitor, and widget settings — all styled with Polaris and served from a normalized database for fast, debuggable queries.

5. Metafield sync

Fitment JSON synced to the custom.vehicle_fitment metafield via GraphQL metafieldsSet, batched in groups for reliable writes to Shopify.

6. Webhook pipeline

products/*, app/uninstalled, and shop/update webhooks processed asynchronously via ActiveJob + Sidekiq, with payload verification for security.

7. Multi-tenant isolation

Per-shop data isolation, encrypted OAuth tokens, and HMAC verification on every request — the standard for Shopify App Store multitenancy.

8. Rate limiting & hardening

rack-attack throttling to protect the App Proxy and API surface from abuse in production.

Architecture & Technology

Rails 7.1 application backend

Ruby on Rails 7.1 on Ruby 3.2. SQLite in development/test and PostgreSQL in production. Standard, up-to-date app architecture with strong separation across models, concerns, jobs, and controllers.

shopify_app OAuth & multitenancy

Standard OAuth 2.0 flow via the shopify_app gem, with per-shop scoping, encrypted tokens, and HMAC-SHA256 verification on App Proxy and webhook traffic.

Normalized local cache

Vehicle, ProductFitment, Shopify metadata, and settings stored in normalized ActiveRecord models (48 YMMTE configurations, 204 verified fitments, 40 products, 8 brands) for lightning-fast storefront queries without hammering Shopify's API.

GraphQL Admin API & Metafields

Data sync runs through Shopify's GraphQL Admin API; fitments are persisted to the custom.vehicle_fitment metafield via metafieldsSet in batches of 25.

Async jobs & Sidekiq

Sync jobs, webhook processors, and cleanup tasks run on Redis-backed Sidekiq with ActiveJob adapters, keeping the request path fast.

Theme App Extension (modern)

The storefront widget is delivered as a Theme App Extension rendered natively in the theme editor — no deprecated ScriptTag API anywhere.

Technology Stack

Layer	Stack
Language / runtime	Ruby 3.2 · Rails 7.1

Shopify integration	shopify_app 22 · shopify_api 14 · GraphQL Admin API
Database	PostgreSQL (production) · SQLite3 (dev/test)
Queuing	Sidekiq 7 + Redis 5 (ActiveJob)
UI styling	Polaris View Components
API & serialization	Oj + ActiveModelSerializers
Testing	RSpec 7 · FactoryBot · WebMock · SimpleCov
Code quality	RuboCop (Rails/RSpec cops) · Bullet (N+1 detection)
Security & hardening	rack-attack rate limiting · HMAC signature checks
Hosting	Fly.io (auto-deploy via GitHub Actions)
CI/CD	GitHub Actions — CI on push, Fly deploy to main

Live Demo Catalog

- **48** **YMMTE vehicle configurations**
- **204** **verified fitments**
- **40** **mapped products**
- **8** **automotive brands**

Key Workflows

Cascading filter flow

Customer picks Year → Make → Model → Trim → Engine; each step narrows available options against the cache, with results returned over the App Proxy API.

Fitment check

Querying the /check_fitment endpoint returns Exact / Does Not Fit / Universal Fit so the UI can badge product cards accurately and instantly.

Metafield sync

Merchant saves fitments; a background job syncs them to Shopify's metafield via GraphQL in batches, with a per-shop sync log for transparency.

Webhook handling

products/update, app/uninstalled, and shop/update events are verified then dispatched to Sidekiq jobs, keeping the storefront cache and tenant data correct.

Built with Free AI Tools — No Paid Tooling

Built with free AI tools. This project was designed, coded, and documented using no-cost AI assistance end to end — no paid AI models, APIs, or commercial tooling were used anywhere in its development. Everything visible here was produced with accessible, free tooling. That is deliberate: it demonstrates the ceiling of what free tooling already achieves, and it means the potential for further growth is massive — with paid, frontier AI models and paid tooling, this same foundation scales into an even richer, production-grade, store-ready solution with ease.

Conclusion

The application is live, deployed, and healthy. Both the storefront shopping experience and the merchant command center are fully functional and can be explored immediately, and the two narrated walkthrough videos demonstrate the complete flow from setup to purchase. Vehicle Selector Pro is ready for further development, App Store submission, or customization to a specific merchant catalog.

Vehicle Selector Pro · github.com/ai-dev-2024/vehicle-selector-pro · vehicle-selector-pro.fly.dev